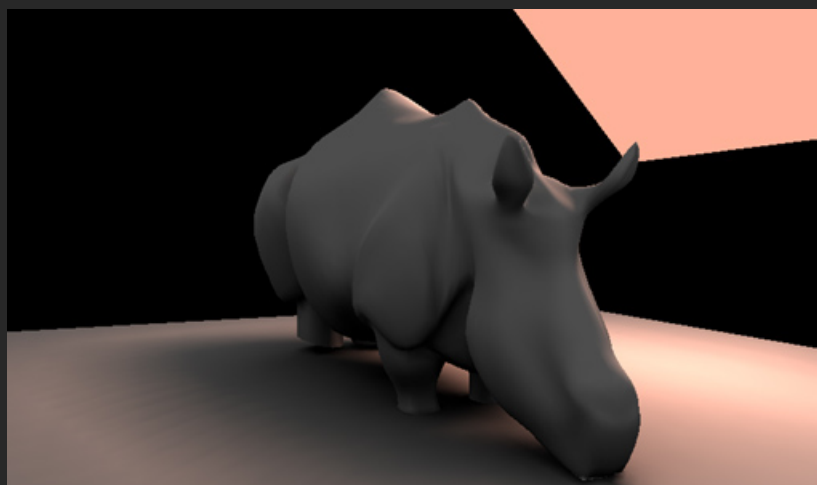


Inigo Quilez :: articles :: simple global illumination - 2004



Just after the *Paradise* 64 kilobytes demo I started to think on adding some GI-like shading to the next intro. After some experiments I decided to add simple Ambient Occlusion to the next release ([195/95/256](#)). That was enough for that production. However the experiments where a bit more complete. This is a quick and simple article about it. It's simple just because the technique itself is ridiculously simple too - simple enough that years later I would use in my 4 kilobyte demo [kinderplomber](#)



one area light



one sky dome light

The idea was to approximate (or fake) global illumination (GI) in very low small 3D scenes. Calculating even a faked variant of GI per pixel was not my best option at that time. Since my scenes were going to be static, I could have done it per texture texel. But I didn't have any texture atlas generation at the time, and I needed a simple technique that could be used in my 64 kilobytes demos - say, all the code/data should fit in 512 bytes maximum. So, I decided to do it per vertex.

So, mesh vertices become the sampling points of the GI, ie, they are light emitters and receivers. We simulate the irradiance emission of a vertex by its color. First, all vertices are black except the vertices belonging to the light sources.

To simulate the first bounce of light, we calculate at each sampling point (vertex) how much light arrives from the emitters. This light gathering process is traditionally done by raycasting the hemisphere around the normal to the surface in each point. However, because we want this to be fast, what we do is to render the scene with a camera in the vertex position and oriented in the direction of the normal. We use a field of view as close to 180 degrees as possible and we accumulate the color of the pixels in the resulting frame buffer to estimate the incoming radiance [a rendering viewport of 32x32 simulates a total of 1024 rays]. This accumulated color will be the irradiance of this vertex for the next pass.

Repeating the process for each vertex completes the simulation of the first bounce of light. Repeating the process allows to simulate more bounces of light, as shown to the right.

The method can of course be applied to as many lights as you want. You can bake them all together or independently. If so, you'll need several color channels per vertex, of course. The advantage is, however, that you can switch the lights on and off separately, which makes the scenes rendered this way a lot more dynamic. A good example is the *Kinderplomber 4*

kilobyte demo that I made in 2007:

This method is not correct of course, because of many reasons. First of all, the pixel color accumulation is simulating the incoming irradiance integration over the emisphere. But, because we are using a simple pinhole camera, we are not correctly weighting the contribution of each ray. The rendering equation (and one's intuition) says that the rays coming more perpendicular to the normal of the vertex should contribute less to the integral (according to a simple cosine term). We could simulate this by applying a spherical projection in the vertex shader, but depending on the needs of the application, we can or not live with the pinhole projection (as it was the case with this images).

A simple extension used in the *195/95/256* intro was to simulate only one bounce of light (so, only direct illumination) but with all the sky dome being a light emitter. This is often called Ambient Occlusion, and can be applied to outdoor scenes, like in the second image in this article. More implementation details [here](#).

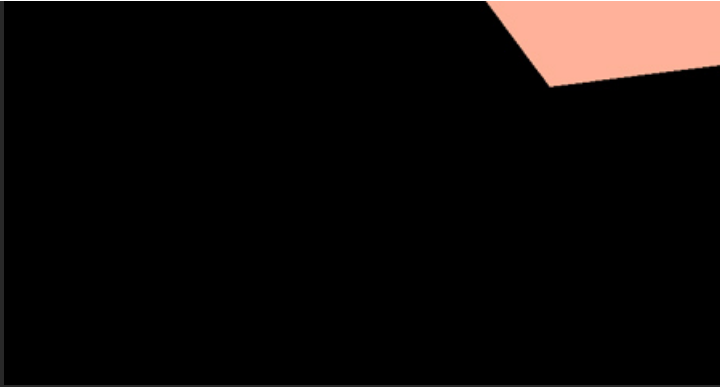


Image 1. Only light sources do emit light.

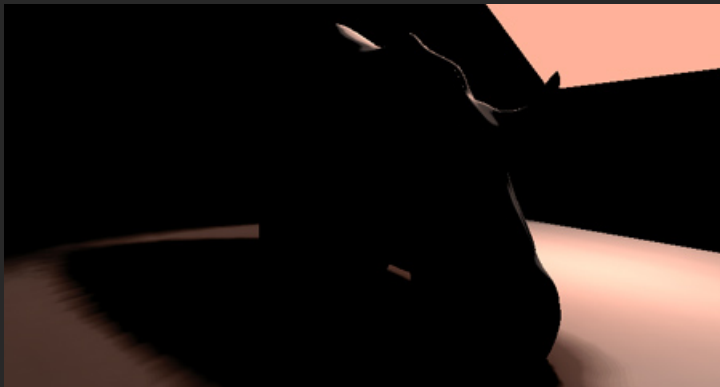


Image 2. First bounce of light. More vertices become emitters.



Image 3. With three bounces we already have a nice image.

inigo quilez - learning computer graphics since 1994